Problem Set 03: Data Wrangling

Owen E. Apple

Last modified on September 02, 2026 13:36:56 Eastern Daylight Time

In this problem set we will practice some of the key data manipulation tasks for describing, summarizing, and working with data. We will specifically review the following functions from the `dplyr` package:

- `select`
- `mutate`
- `summarize`
- `arrange`
- `filter`
- `group_by`

In addition we will review how to save objects using the `<-` assignment operator.

The following code loads the necessary packages for this problem set:

R Code

```
library(ggplot2)
library(dplyr)
```

The Data

The following code chunk loads the data set `txhousing` and displays the data using `glimpse`:

R Code

```
data(txhousing)
glimpse(txhousing)
```

```
Rows: 8,602
```

```
Columns: 9
$ city      <chr> "Abilene", "Abilene", "Abilene", "Abilene", "Abilene", "Abil~
$ year      <int> 2000, 2000, 2000, 2000, 2000, 2000, 2000, 2000, 2000, 2000, ~
$ month     <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 1, 2, 3, 4, 5, 6, 7, ~
$ sales     <dbl> 72, 98, 130, 98, 141, 156, 152, 131, 104, 101, 100, 92, 75, ~
$ volume    <dbl> 5380000, 6505000, 9285000, 9730000, 10590000, 13910000, 1263~
$ median    <dbl> 71400, 58700, 58100, 68600, 67300, 66900, 73500, 75000, 6450~
$ listings  <dbl> 701, 746, 784, 785, 794, 780, 742, 765, 771, 764, 721, 658, ~
$ inventory <dbl> 6.3, 6.6, 6.8, 6.9, 6.8, 6.6, 6.2, 6.4, 6.5, 6.6, 6.2, 5.7, ~
$ date      <dbl> 2000.000, 2000.083, 2000.167, 2000.250, 2000.333, 2000.417, ~
```

```
#
#txhousing <- txhousing[sample(1:802),]
```

These data are about housing in Texas. Each row is monthly data for a given city in Texas in a given year. There are multiple years of data for each city.

Problem 1

Examine `txhousing` in the data viewer. You can accomplish this two different ways: A) click on the name of the data in the Environment pane, or b) type `View(txhousing)` in the **console**. What is the last city listed in the data set (in row 8602)? Use `R` code to programatically find the last city in `txhousing`.

Problem 1 Answers

```
# Type your code and comments inside the code chunk
tail(txhousing, n = 1)$city
```

```
[1] "Wichita Falls"
```

- The last city listed in the data set `txhousing` is **Wichita Falls**.

Problem 2

Examine the variable descriptions by typing `?txhousing` in the **console**. What is the `listings` variable in this data set?

Problem 2 Answers

- In this data set, the double variable `listings` is the number of total active listings.

Data Wrangling Review

`select()`

Sometimes we want to pull out or extract just one or two columns of data. The following code will extract only the columns in the data set for the variables `sales` and `volume`.

```
txhousing |>
  select(sales, volume)
```

The `|>` symbol is called the **piping** operator. Here, it takes the `txhousing` **data frame** and "pipes" or feeds it into the `select` function. You can think of the `|>` symbol as the word "then".

Note that an assignment operator (`<-`) was not used in the code; consequently the selected values are not saved. In the following code, the results are saved in a data frame **ALSO** called `txhousing`. By putting `-` in front of the `date` variable `R` selects all variables **except** the `date` variable.

R Code

```
txhousing <- txhousing |>
  select(-date)
head(txhousing)
```

```
# A tibble: 6 x 8
  city     year month sales   volume median listings inventory
  <chr>   <int> <int> <dbl>    <dbl>  <dbl>    <dbl>     <dbl>
1 Abilene  2000     1    72  5380000  71400      701       6.3
2 Abilene  2000     2    98  6505000  58700      746       6.6
3 Abilene  2000     3   130  9285000  58100      784       6.8
4 Abilene  2000     4    98  9730000  68600      785       6.9
5 Abilene  2000     5   141 10590000  67300      794       6.8
6 Abilene  2000     6   156 13910000  66900      780       6.6
```

If you examine `txhousing` in the data viewer, the `date` variable is no longer included.

`filter()`

The filter function allows you to pull out just the **rows** (cases or observations) you want, based on some criteria in **one of the columns**.

Imagine we wanted to reduce the data set to include data for only 2012 in the city of Austin. The code chunk below filters the `txhousing` to only include rows in which the year is 2012 **and** the city is Austin. The results are saved in a new data frame called `austin_12`.

R Code

```
austin_12 <- txhousing |>
  filter(year == 2012 & city == "Austin")
  #Or filter(year == 2012, city == "Austin")
head(austin_12)
```

```
# A tibble: 6 x 8
  city    year month sales    volume median listings inventory
  <chr>  <int> <int> <dbl>     <dbl>  <dbl>    <dbl>     <dbl>
1 Austin  2012     1  1182 265821275 177400     7432       4.2
2 Austin  2012     2  1415 353527608 191600     7738       4.3
3 Austin  2012     3  2083 533800484 198600     8186       4.5
4 Austin  2012     4  2128 563288160 207400     8239       4.5
5 Austin  2012     5  2611 705383898 210200     8465       4.5
6 Austin  2012     6  2837 791281075 216000     8641       4.5
```

Note

Note that we use `==` to identify the desired criteria.

What if we wanted to restrict our data set to only years before 2004 and the City of Austin? Below we use the `<` symbol to accomplish this. Note we did not **SAVE** these results in a new data frame...so no new data frame showed up in our Environment pane, but the results print out immediately below the code chunk.

R Code

```
txhousing |>
  filter(year < 2004, city == "Austin") |>
  head()
```

```
# A tibble: 6 x 8
  city    year month sales    volume median listings inventory
  <chr>  <int> <int> <dbl>     <dbl>  <dbl>    <dbl>     <dbl>
1 Austin  2000     1  1025 173053635 133700     3084         2
2 Austin  2000     2  1277 226038438 134000     2989         2
3 Austin  2000     3  1603 298557656 136700     3042         2
```

```
4 Austin  2000     4  1556 289197960 136900     3192       2.1
5 Austin  2000     5  1980 393073774 144700     3617       2.3
6 Austin  2000     6  1885 368290072 148800     3799       2.4
```

What if we wanted to use multiple cities? Below we use the | symbol to indicate that the city could be Austin **OR** Abilene. In this case, we **saved** these results as a new data frame called `aust_ab` that appears in your Environment pane.

R Code

```
aust_ab <- txhousing |>
  filter(city == "Austin" | city == "Abilene")
head(aust_ab)
```

```
# A tibble: 6 x 8
  city     year month sales   volume median listings inventory
  <chr>   <int> <int> <dbl>    <dbl>  <dbl>    <dbl>     <dbl>
1 Abilene  2000     1    72  5380000  71400      701       6.3
2 Abilene  2000     2    98  6505000  58700      746       6.6
3 Abilene  2000     3   130  9285000  58100      784       6.8
4 Abilene  2000     4    98  9730000  68600      785       6.9
5 Abilene  2000     5   141 10590000  67300      794       6.8
6 Abilene  2000     6   156 13910000  66900      780       6.6
```

```
tail(aust_ab)
```

```
# A tibble: 6 x 8
  city    year month sales     volume median listings inventory
  <chr>  <int> <int> <dbl>      <dbl>  <dbl>    <dbl>     <dbl>
1 Austin  2015     2  1978  595625521 245300     5733       2.2
2 Austin  2015     3  2677  885779822 253900     5906       2.3
3 Austin  2015     4  2801  931744729 270300     6560       2.5
4 Austin  2015     5  2999 1026501450 271200     7009       2.7
5 Austin  2015     6  3301 1086689918 270200     7419       2.8
6 Austin  2015     7  3466 1150381553 264600     7913       3
```

`mutate()`

The mutate function can add new columns (variables) to a data frame. For instance, the following will add a new column to the data called `vol_100k` that expresses volume in units of $100,000.

R Code

```
txhousing <- txhousing |>
  mutate(vol_100k = volume/100000)
head(txhousing)
```

```
# A tibble: 6 x 9
  city     year month sales   volume median listings inventory vol_100k
  <chr>   <int> <int> <dbl>    <dbl>  <dbl>    <dbl>     <dbl>    <dbl>
1 Abilene  2000     1    72  5380000  71400      701       6.3     53.8
2 Abilene  2000     2    98  6505000  58700      746       6.6     65.0
3 Abilene  2000     3   130  9285000  58100      784       6.8     92.8
4 Abilene  2000     4    98  9730000  68600      785       6.9     97.3
5 Abilene  2000     5   141 10590000  67300      794       6.8    106.
6 Abilene  2000     6   156 13910000  66900      780       6.6    139.
```

Note that we **SAVED** these results in new data frame called `txhousing`. This therefore **overwrote** the old `txhousing` data frame with a new version that contains this column. You can open the `txhousing` data frame in the viewer to confirm that it now contains this new column.

`summarize()`

One of the first tasks in data analysis is often to get descriptive statistics that help to understand the central tendency and variability in the data. The `summarize()` command can take a column of data, and reduce it to a summary statistic.

For instance, the code below uses the `austin_12` data set made earlier to calculate the mean monthly number of `sales` in Austin in 2012.

```
austin_12 |>
  summarize(x_bar_sales = mean(sales))
```

This code tells `R` to calculate the `mean` of the variable `sales`, and to save the results in a variable called `x_bar_sales`.

You can also calculate multiple summary statistics at once, and even for multiple variables. Below we also calculate a standard deviation `sd()` of `sales`, a minimum `min()` of the `volume` variable, a maximum `max()` of the `volume` variable, etc. The `n()` calculates sample size…or the number of rows/ cases in the data frame.

R Code

```
austin_12 |>
  summarize(x_bar_sales = mean(sales),
            sd_sales = sd(sales),
            min_vol = min(volume),
            max_vol = max(volume),
            mdn_list = median(listings),
            iqr_list = IQR(listings),
            sample_size = n()) -> ans1
kable(ans1)
```

x_bar_sales	sd_sales	min_vol	max_vol	mdn_list	iqr_list	sample_size
2126.75	500.8361	265821275	791281075	7925	948.75	12

Note that the names of the elements you calculate are user defined, like `xbar_sales`, `min_vol`, and `mdn_list`. You could customize these names as you like (but don't use spaces in your names).

`arrange()`

You just determined that the maximum volume of monthly sales in Austin in 2012 was a total of $791,281,075 … but what if you wanted to know **WHAT MONTH** that occurred in?

R Code

```
austin_12 |>
  arrange(desc(volume)) -> ans2
head(ans2, n = 3) |>
  kable()
```

city	year	month	sales	volume	median	listings	inventory
Austin	2012	6	2837	791281075	216000	8641	4.5
Austin	2012	7	2604	718755768	211000	8519	4.3
Austin	2012	8	2647	708540314	205100	8112	4.0

The above code tells `R` to `arrange` the rows in the data set based on the `volume` column and to do so in `desc`ending order. Consequently, the row with the $791,281,075 in sales is shown at the top. We can see that this `volume` occurred in the 6^{th} month (June).

`group_by()`

Sometimes we also want to calculate summary statistics across different levels of another variable. For instance, here we find the average number of monthly sales that occurred in Abilene and Austin across all years in the data set. Note that we **use the `aust_ab` data frame** we created earlier, to restrict our analysis to those two cities.

R Code

```
aust_ab |>
  group_by(city) |>
  summarize(x_bar_sales = mean(sales)) -> results
results |>
  kable()
```

city	x_bar_sales
Abilene	150.4866
Austin	1996.6898

From the results we can see that there were an average of 150.5 sales per month in Abilene, and 1996.7 sales per month in Austin.

We can give `R` multiple variables to group by. For instance, the following code returns the mean sales for each month in each city averaged across all the years.

R Code

```
aust_ab |> group_by(city, month) |>
  summarize(x_bar_sales = mean(sales)) -> results_ab
results_ab |>
  head() |>
  kable()
```

city	month	x_bar_sales
Abilene	1	96.3125
Abilene	2	121.0000
Abilene	3	151.3750
Abilene	4	159.8750
Abilene	5	177.8750
Abilene	6	190.3125

The mean number of sales for Abilene in January (across all years) was 96.3 homes.

Independent Practice

Basic Syntax

This first set of questions will help you practice basic syntax.

Problem 3

Write a code chunk to remove the inventory variable. Save the results in a data frame called `txhousing`. Confirm the variable `inventory` has been removed.

Problem 3 Answers

```
# Type your code and comments inside the code chunk
txhousing <- txhousing |>
  select(-inventory)
names(txhousing)
```

```
[1] "city"     "year"     "month"    "sales"    "volume"   "median"   "listings"
[8] "vol_100k"
```

- After running the code above, the inventory variable was removed.

Problem 4

Make a data set called `dallas_sub` that includes data only from the city of Dallas in 2012 and 2013. Verify that the dimensions of `dallas_sub` are 24 by 8. Show the first six rows of `dallas_sub`. Explain why the following code does not give the requested answer:

```
txhousing |>
  filter(city =="Dallas" & year == 2012 | year == 2013) |> dim()
```

Problem 4 Answers

```
# Type your code and comments inside the code chunk
dallas_sub <- txhousing |>
  filter(city == "Dallas", year == 2012 | year == 2013)
dim(dallas_sub)
```

```
[1] 24  8
```

```
head(dallas_sub)
```

```
# A tibble: 6 x 8
  city    year month sales     volume median listings vol_100k
  <chr>  <int> <int> <dbl>      <dbl>  <dbl>    <dbl>    <dbl>
1 Dallas  2012     1  2555  509458081 150800    16721    5095.
2 Dallas  2012     2  3085  634067291 157100    17173    6341.
3 Dallas  2012     3  4068  898320563 167300    17433    8983.
4 Dallas  2012     4  4291  983333297 168700    17632    9833.
5 Dallas  2012     5  5004 1175419749 175100    17726   11754.
6 Dallas  2012     6  5196 1209024869 177900    17587   12090.
```

- The reason the code given in Problem 4 is incorrect is because of this use of `&` in the code block. This causes the code to group `Dallas` and `2012` together to display all 2012 columns for Dallas or display `2013` columns for every city. Replacing `&` with `,` fixes this issue.

Problem 5

Add a column to the `dallas_sub` data set called `prct_sold` that calculates the percentage of `listings` that were `sold` (`sales/listings * 100`). Be sure to **save** the results into a data frame called `dallas_sub`. Display the last six rows of `dallas_sub`.

Problem 5 Answers

```
# Type your code and comments inside the code chunk
dallas_sub <- dallas_sub |>
  mutate(prct_sold = (sales/listings * 100))
dim(dallas_sub)
```

```
[1] 24  9
```

```
tail(dallas_sub)
```

```
# A tibble: 6 x 9
  city    year month sales     volume median listings vol_100k prct_sold
  <chr>  <int> <int> <dbl>      <dbl>  <dbl>    <dbl>    <dbl>     <dbl>
1 Dallas  2013     7  6322 1598961793 199200    13685   15990.      46.2
2 Dallas  2013     8  6209 1570480866 198000    13694   15705.      45.3
```

```
3 Dallas  2013     9  4941 1181585876 188800    13145   11816.      37.6
4 Dallas  2013    10  4593 1105924279 189200    12728   11059.      36.1
5 Dallas  2013    11  3964  968261356 186600    11895    9683.      33.3
6 Dallas  2013    12  4212 1071755610 194900    10531   10718.      40.0
```

Problem 6

Calculate the **average** percentage of listings that were sold in Dallas **in each month across the years** based on your `dallas_sub` data set. Save the results of the calculation in a data frame called `dallas_summary` with the results stored in `mean_prct_sold`. Display the results of `dallas_summary`.

Problem 6 Answers

```r
# Type your code and comments inside the code chunk
dallas_summary <- dallas_sub |>
  group_by(month) |>
  summarize(mean_prct_sold = mean(prct_sold))
kable(dallas_summary)
```

month	mean_prct_sold
1	20.54462
2	23.47328
3	32.24472
4	34.46954
5	38.20168
6	37.19150
7	37.13028
8	38.51543
9	31.77096
10	32.08208
11	30.59329
12	35.47333

Problem 7

Arrange the `dallas_summary` in `desc`ending order based on the average percentage of listings that were sold in Dallas, so you can see **which month** had the greatest percentage of houses sold in Dallas on average from 2012-2013. You do not need to save the results.

Problem 7 Answers

```
# Type your code and comments inside the code chunk
dallas_summary |>
  arrange(desc(mean_prct_sold)) |>
  kable()
```

month	mean_prct_sold
8	38.51543
5	38.20168
6	37.19150
7	37.13028
12	35.47333
4	34.46954
3	32.24472
10	32.08208
9	31.77096
11	30.59329
2	23.47328
1	20.54462

More Advanced Wrangling

Please answer the following questions with text and/or code where appropriate. You may have to use multiple `dplyr` functions to answer each question. Think through the steps of how to get to the answer you are trying to find.

Problem 8

Run the following code chunk. Study the code, and the output. Explain in your own words what this code chunk calculated. Specifically, compare this code to the code in Problems 4 through 7.

Problem 8 Answers

```
txhousing |>
  filter(year == 2012 | year == 2013, city == "Dallas") |>
  mutate(prct_sold = sales/listings *100) |>
  group_by(month) |>
  summarize(mean_prct_sold = mean(prct_sold)) |>
  arrange(desc(mean_prct_sold)) |>
  kable()
```

month	mean_prct_sold
8	38.51543
5	38.20168
6	37.19150
7	37.13028
12	35.47333
4	34.46954
3	32.24472
10	32.08208
9	31.77096
11	30.59329
2	23.47328
1	20.54462

- The code chunk above seems to have calculated the mean percent of listings sold in Dallas in each month across the year 2012 and 2013, arranged from largest to smallest. Compared to code from the other problems, you can see clearly the `filter()` function being used to only use data from Dallas in the years of 2012 and 2013. This is similar to how `filter()` was used in **Problem 4**. Next, `mutate()` is used to create a new variable called `prct_sold`, which is the percent of listed houses that were sold in each month of 2012 and 2013. The use of the `mutate()` function is similar to how it was used in **Problem 5**. Next, the `group_by()` and `summarize()` functions were used to first group together the `prct_sold` by each month with `group_by()`, and then to find the average percent sold by making the new variable `mean_prct_sold` using the `summarize()` function. This is similar to how these functions were used in **Problem 6**. Lastly, the `arrange()` function is used to arrange the `mean_prct_sold` variable created by the `summarize()` function in `desc`ending order, similar to how `arrange()` is used in **Problem 7**.

Problem 9

In January of 2015, what city had the fewest houses listed for sale? Report the city and the number of houses said city had listed for sale using inline `R` code.

Problem 9 Answers

```
# Type your code and comments inside the code chunk
txhousing |>
  filter(year == 2015, month == 1) |>
  arrange(listings) |>
  select(city, listings) |>
  head(n=1) |>
  kable()
```

city	listings
San Marcos	85

- The Texas city with the fewest houses listed for sale in January of 2015 was San Marcos, with only 85 listings.

Problem 10

In 2012, in which month were the most houses sold in Texas? Report the month were the most houses sold in Texas and the number of houses sold for that month using inline `R` code.

Problem 10 Answers

```
# Type your code and comments inside the code chunk
txhousing |>
  filter(year == 2012) |>
  group_by(month) |>
  summarize(month_sales = sum(sales)) |>
  select(month, month_sales) |>
  arrange(desc(month_sales)) |>
  head(month_sales, n=1) |>
  kable()
```

month	month_sales
8	29995

- The 8th of the year was the month with the most houses sold in Texas. There was 29,995 sales that month.

Problem 11

Generate a **single** table that shows the total number of houses sold in **Austin** in **2000 and 2001** (total over the entire period), & the total number of houses sold in **Dallas** in **2000 and 2001** (total over the entire period). This calculation requires a number of steps, so it might help you to first write out on paper the different steps you will need to take. That will help you set out a "blueprint" for tackling the problem. **Hint**: recall the `sum()` function can add values.

Problem 11 Answers

```
# Type your code and comments inside the code chunk
txhousing |>
  filter(city == "Austin"| city == "Dallas", year == 2000 | year == 2001) |>
  group_by(city) |>
  summarize(total_sold = sum(sales)) |>
  kable()
```

city	total_sold
Austin	37013
Dallas	92438

Turning in Your Work

You will need to make sure you commit and push all of your changes to the github education repository where you obtained the lab.

Tip

- Make sure you **render a final copy with all your changes** and work.
- Look at your final html file to make sure it contains the work you expect and is

formatted properly.

Logging out of the Server

There are many statistics classes and students using the Server. To keep the server running as fast as possible, it is best to sign out when you are done. To do so, follow all the same steps for closing Quarto document:

Tip

- Save all your work.
- Click on the orange button in the far right corner of the screen to quit `R`
- Choose **don't save** for the **Workspace image**
- When the browser refreshes, you can click on the sign out next to your name in the top right.
- You are signed out.

```
sessionInfo()
```

```
R version 4.6.1 (2026-06-24)
Platform: x86_64-redhat-linux-gnu
Running under: Red Hat Enterprise Linux 9.8 (Plow)

Matrix products: default
BLAS/LAPACK: FlexiBLAS OPENBLAS-OPENMP;  LAPACK version 3.9.0

locale:
 [1] LC_CTYPE=en_US.UTF-8       LC_NUMERIC=C
 [3] LC_TIME=en_US.UTF-8        LC_COLLATE=en_US.UTF-8
 [5] LC_MONETARY=en_US.UTF-8    LC_MESSAGES=en_US.UTF-8
 [7] LC_PAPER=en_US.UTF-8       LC_NAME=C
 [9] LC_ADDRESS=C               LC_TELEPHONE=C
[11] LC_MEASUREMENT=en_US.UTF-8 LC_IDENTIFICATION=C

time zone: America/New_York
tzcode source: system (glibc)

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base
```

```
other attached packages:
 [1] scales_1.4.0    lubridate_1.9.5 forcats_1.0.1   stringr_1.6.0
 [5] dplyr_1.2.1     purrr_1.2.2     readr_2.2.0     tidyr_1.3.2
 [9] tibble_3.3.1    ggplot2_4.0.3   tidyverse_2.0.0 knitr_1.51

loaded via a namespace (and not attached):
 [1] gtable_0.3.6       jsonlite_2.0.0     compiler_4.6.1     tidyselect_1.2.1
 [5] dichromat_2.0-0.1  yaml_2.3.12        fastmap_1.2.0      R6_2.6.1
 [9] generics_0.1.4     pillar_1.11.1      RColorBrewer_1.1-3 tzdb_0.5.0
[13] rlang_1.2.0        utf8_1.2.6         stringi_1.8.7      xfun_0.57
[17] S7_0.2.2           otel_0.2.0         timechange_0.4.0   cli_3.6.6
[21] withr_3.0.2        magrittr_2.0.5     digest_0.6.39      grid_4.6.1
[25] rstudioapi_0.18.0  hms_1.1.4          lifecycle_1.0.5    vctrs_0.7.3
[29] evaluate_1.0.5     glue_1.8.1         farver_2.1.2       rmarkdown_2.31
[33] tools_4.6.1        pkgconfig_2.0.3    htmltools_0.5.9
```